Documento Técnico: Gestión de Liga de Básquetbol (PHP MVC)

- **Extensión:** 718 palabras de prosa (requisito: entre 500 y 800)
- **Autor:** Proyecto de Aprendizaje
- **Última actualización:** 2026-10-30

1. Arquitectura

El sistema implementa una arquitectura **MVC ligero** en PHP vanilla, sin frameworks ni gestor de dependencias. Toda petición entra por un único front controller, `public/index.php`, que inicializa sesión, cookies y manejador global de errores antes de delegar; el `index.php` de la raíz solo lo incluye para permitir entrar sin escribir `/public/`. `app/core/Router.php` interpreta `controller`, `action` y parámetros de la URL, y los valida contra una lista blanca declarada en la constante `RUTAS`. Las capas son: **Modelos** (datos y reglas de negocio), **Vistas** (plantillas PHP con layout compartido), **Controladores** (coordinación), **Core** (clases base reutilizables) y **Config** (constantes). La raíz del proyecto expone únicamente `index.php` y los recursos de `public/`: el `.htaccess` raíz reescribe las peticiones al front controller y devuelve 403 para `app/`, `database/`, `doc/`, `vault/`, `tests/` y `.git/`, de modo que ni los scripts SQL con los hashes de las contraseñas de prueba ni el historial de Git quedan descargables.

2. Componentes Clave

`Database.php` implementa el patrón *singleton* sobre PDO con `ERRMODE_EXCEPTION`, `FETCH_ASSOC` y `utf8mb4`. `Model.php` aporta `obtenerTodos()`, `obtenerPorId()`, `eliminar()` y `obtenerUltimoId()`; cada modelo concreto añade sus propios `crear()` y `actualizar()`. `Controller.php` resuelve la vista con `extract()` y el layout, e implementa `requiereLogin()`. `helpers.php` centraliza funciones sin clase: escapado, tokens CSRF, mensajes flash, redirecciones y validación. `LimitadorIntentos.php` cuenta los intentos de login fallidos en el servidor, por IP y cuenta, para frenar la fuerza bruta. Los modelos `Usuario`, `Equipo`, `Jugador` y `Partido` heredan de `Model`; los controladores `AuthController`, `EquiposController`, `JugadoresController`, `PartidosController` y `EstadisticasController` coordinan los cinco módulos exigidos.

3. Funcionalidades Desarrolladas

Autenticación: registro con validación de email, coincidencia de contraseñas y longitud mínima; inicio de sesión con `password_verify()`, regeneración de identificador de sesión y bloqueo tras cinco intentos fallidos. **Equipos, jugadores y partidos:** CRUD completo con listados, asignación de jugadores a equipos y registro de encuentros con marcador y resultado calculado. **Estadísticas:** tabla de posiciones (PJ, PG, PE, PP, GF, GC, DIF) ordenada por victorias y diferencia, más los últimos diez resultados.

4. Código Fuente Relevante

El enrutamiento contrasta la URL con una lista blanca antes de cargar nada, y solo después comprueba que el método sea invocable:

```
if (!array_key_exists($controlador, self::RUTAS)) {

    $this->error('Controlador no permitido. Vuelve a la página principal.');

}



$permitidas = self::RUTAS[$controlador];

if (!in_array($accion, $permitidas, true)) {

    $this->error('Acción no permitida para este controlador.');

}
```

El borrado exige POST y token válido antes de tocar la base de datos:

```
exigirPost('controller=Equipos&action=index');
```

Las consultas usan siempre sentencias preparadas con parámetros vinculados, nunca concatenación de variables.

5. Resolución de Problemas

La primera versión delegaba toda la validación en el navegador y documentaba sus carencias. La revisión final las corrigió en el servidor. El problema más serio era el **borrado de un equipo con jugadores**: la restricción `ON DELETE RESTRICT` lanzaba una

excepción sin capturar y el usuario veía una pantalla vacía. Se añadió una comprobación previa de dependencias y un manejador global que traduce los códigos de MySQL en mensajes comprensibles, sin exponer el texto original. También se detectaron y corrigieron un XSS reflejado en el mensaje del enrutador, la desviación de redirecciones mediante la cabecera `Host`, campos enviados como array que provocaban errores internos, dorsales duplicados y posiciones fuera de catálogo.

Dos detalles merecen mención: `DateTime::createFromFormat()` acepta fechas inexistentes como el 31 de febrero, por lo que se validan sus avisos; y `rowCount()` cuenta filas *cambiadas*, no *encontradas*, de modo que guardar un registro sin modificarlo se confundía con un identificador inexistente.

6. Decisiones de Diseño

Se eligió PHP vanilla para comprender los mecanismos sin abstracciones ocultas, y una lista blanca en el enrutador por seguridad: sólo se cargan los módulos declarados en ella. `posicion` es texto validado contra un catálogo en lugar de `ENUM`, para admitir valores nuevos sin alterar el esquema. La contraseña se almacena con el hash por defecto de PHP y las sesiones exigen cookies `HttpOnly`, `SameSite` y `Secure` bajo HTTPS. El despliegue admite dos formas: apuntar el `DocumentRoot` a `public/` (la más segura) o copiar la carpeta completa en `htdocs/` y entrar por su raíz, donde `index.php` delega en `public/index.php` y el `.htaccess` bloquea el resto del proyecto. El historial Git se mantiene con commits atómicos y mensajes descriptivos, y cada bloque de trabajo se desarrolla en su propia rama, que se integra en `main` con un commit de fusión. Una suite de 97 comprobaciones automatizadas verifica el comportamiento y restaura los datos de prueba al terminar.

Cumplen los requerimientos: los cinco módulos funcionales, control de versiones con commits atómicos, este documento en `/doc/practica_liga.md` e historial `.git` intacto en el paquete de entrega.